

# Algoritmi e Strutture Dati

## Ricerca locale

Alberto Montresor

Università di Trento

2026/08/15

This work is licensed under a Creative Commons  
Attribution-ShareAlike 4.0 International License.



# Sommario

- 1 Introduzione
- 2 Flusso massimo
  - Rete di flusso
  - Flusso
  - Problema
  - Algoritmo
- 3 Dimostrazione di correttezza
- 4 Complessità
- 5 Oltre il Flusso Massimo
  - Abbinamento grafi bipartiti

# Ricerca locale

Se si conosce una soluzione ammissibile (non necessariamente ottima) ad un problema di ottimizzazione, si può cercare una soluzione migliore nelle "vicinanze" di quella precedente.

Si continua così fino a quando non si trova più un miglioramento locale.

---

`localSearch()`

---

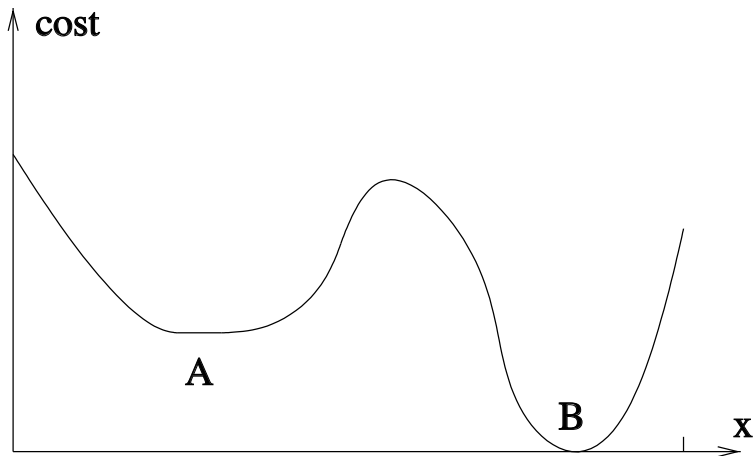
*solution* = una soluzione ammissibile del problema

**while**  $\exists$  *candidate*  $\in$  `neighborhood`(*solution*) **migliore di** *solution* **do**  
    *solution* = *candidate*

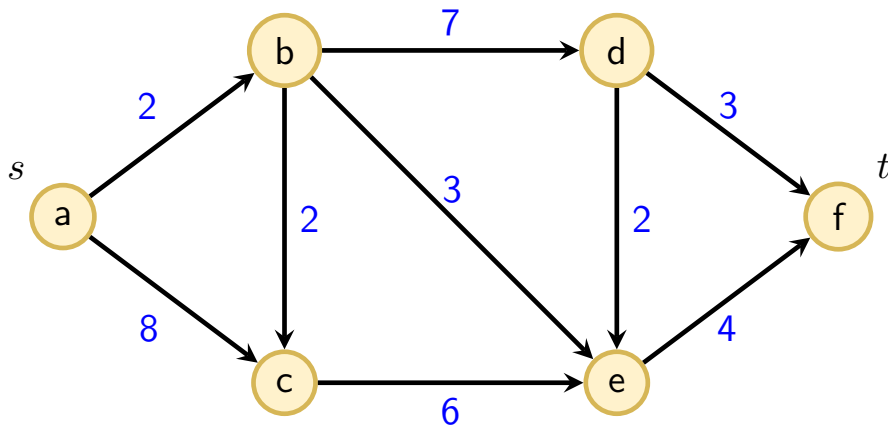
**return** *solution*

---

# Ricerca locale



# Rete di flusso, sorgente, pozzo, capacità



# Rete di flusso

## Definizione

Una **rete di flusso**  $G = (V, E, s, t, c)$  è data da:

- un grafo orientato  $G = (V, E)$
- un nodo  $s \in V$  detto **sorgente**
- un nodo  $t \in V$  detto **pozzo**
- una funzione di **capacità**  $c : V \times V \rightarrow \mathbb{R}^{\geq 0}$ ,  
tale che  $(x, y) \notin E \Rightarrow c(x, y) = 0$ .

## Assunzioni

- Per ogni nodo  $x \in V$ , esiste un cammino  $s \rightsquigarrow x \rightsquigarrow t$  da  $s$  a  $t$  che passa per  $x$ .
- Possiamo ignorare i nodi che non godono di questa proprietà

# Flusso

## Flusso

Un **flusso** in  $G$  è una funzione  $f : V \times V \rightarrow \mathbb{R}$  che soddisfa le seguenti proprietà:

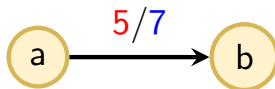
- **Vincolo sulla capacità:**  $\forall x, y \in V, f(x, y) \leq c(x, y)$
- **Antisimmetria:**  $\forall x, y \in V, f(x, y) = -f(y, x)$
- **Conservazione del flusso:**  $\forall x \in V - \{s, t\}, \sum_{y \in V} f(x, y) = 0$

# Flusso

## Vincolo sulla capacità

Il flusso non deve eccedere la capacità sull'arco.

$$\forall x, y \in V : f(x, y) \leq c(x, y)$$



# Flusso

## Simmetria opposta

Il flusso che attraversa un arco in direzione  $(x, y)$  è l'opposto del flusso che attraversa l'arco in direzione  $(y, x)$ .

$$\forall x, y \in V : f(x, y) = -f(y, x)$$



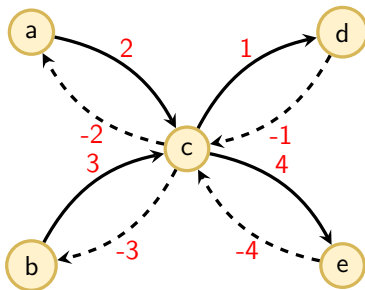
Trucchetto matematico che semplificherà molte dimostrazioni.  
Un valore positivo  $f(x, y)$  rappresenta il flusso netto da  $x$  a  $y$ .  
Un valore negativo indica flusso netto nella direzione opposta.

# Flusso

## Conservazione del flusso

Per ogni nodo diverso da sorgente e pozzo, la somma dei flussi entranti deve essere uguale alla somma dei flussi uscenti.

$$\forall x \in V - \{s, t\} : \sum_{y \in V} f(x, y) = 0$$



# Definizioni

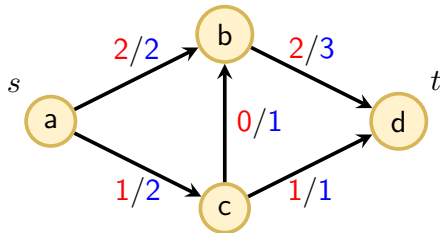
## Valore del flusso

Il **valore di un flusso**  $f$  è definito come:

$$|f| = \sum_{x \in V} f(s, x)$$

ovvero come la quantità di flusso uscente da  $s$ .

$$|f| = 3$$



## Legenda

- Flusso in **rosso**
- Capacità iniziale in **blu**

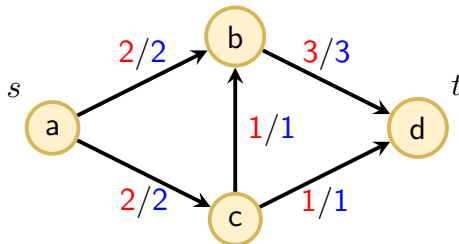
# Problema

## Flusso massimo

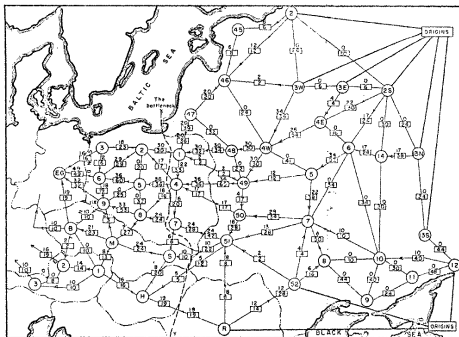
Data una rete  $G = (V, E, s, t, c)$ , trovare un flusso che abbia valore massimo fra tutti i flussi associabili alla rete.

$$|f^*| = \max\{|f|\}$$

$$|f^*| = 4$$



# Flusso massimo nella guerra fredda



T. Harris, F. Ross, *Fundamentals of a method for evaluating rail net capacities*. Research Memorandum RM-1573, The RAND Corporation, Santa Monica, California, 1955

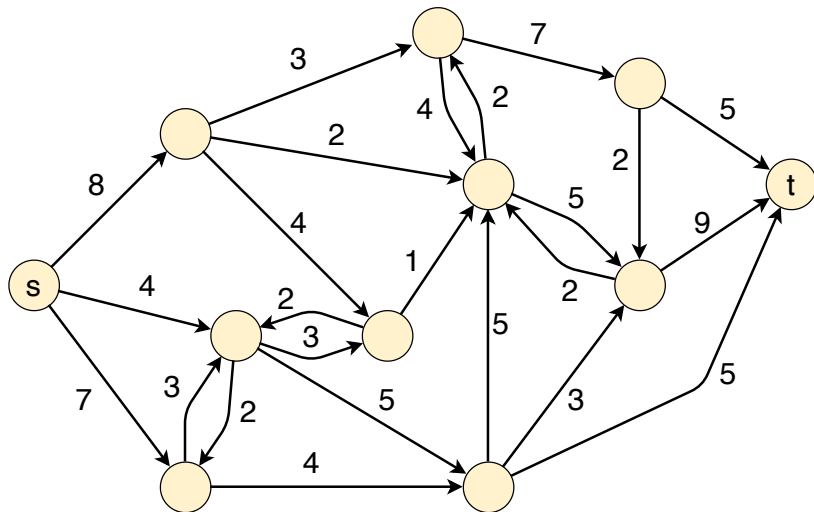
Rete ferroviaria dell'Unione Sovietica occidentale e dell'Europa Orientale.

$|V| = 44, |E| = 105$ .

Flusso massimo di  $163 \cdot 10^6$  kg.

Alexander Schrijver. On the history of the transportation and maximum flow problems

## Esercizio - Trovate il flusso massimo



<https://disi.unitn.it/~montreso/asd/material/flow/flusso-esempio1.pdf>

## Flusso massimo come ricerca locale

Nel problema del flusso massimo, una soluzione ammissibile è un flusso  $f$  che rispetta capacità, antisimmetria e conservazione.

Una **mossa locale** consiste nel modificare  $f$  lungo alcuni archi, in modo da aumentare il valore totale del flusso senza violare i vincoli.

Il punto cruciale sarà capire quali modifiche sono ancora possibili a partire dal flusso corrente.

Nel caso del flusso massimo, quando nessuna modifica migliorativa è più possibile, il flusso ottenuto è ottimo.

# Metodo delle reti residue

## Algoritmo, informale

- Si memorizza un flusso "corrente"  $f$ , inizialmente nullo
  - Si ripetono le operazioni seguenti:
    - Si "sottrae" il flusso attuale dalla rete iniziale, ottenendo una **rete residua**
    - Si cerca un **flusso  $g$**  all'interno della rete residua
    - Si somma  $g$  ad  $f$
- fino a quando non è più possibile trovare un flusso positivo  $g$

## Output

È possibile dimostrare che questo approccio restituisce un flusso massimo

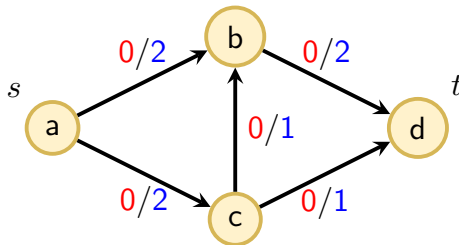
# Definizioni

## Flusso nullo

Definiamo **flusso nullo** la funzione  $f_0 : V \times V \rightarrow \mathbb{R}$  tale che:

$$\forall x, y \in V : f_0(x, y) = 0$$

$$|f_0| = 0$$

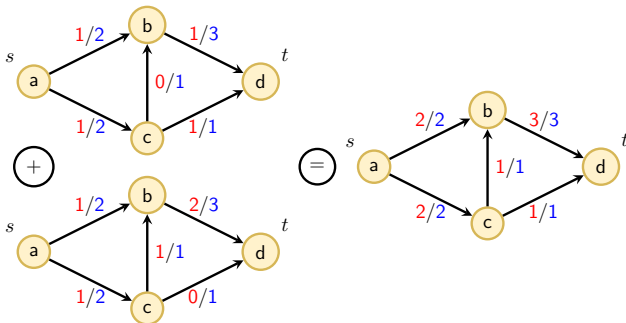


# Definizioni

## Somma di flussi

Per ogni coppia di funzioni  $f_1, f_2 : V \times V \rightarrow \mathbb{R}$ , definiamo la **funzione somma**  $f_t = f_1 + f_2$  come:

$$\forall x, y \in V : f_t(x, y) = f_1(x, y) + f_2(x, y)$$

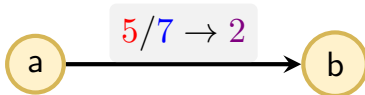


# Definizioni

## Capacità residua

Definiamo **capacità residua** di un flusso  $f$  in una rete  $G = (V, E, s, t, c)$  una funzione  $c_f : V \times V \rightarrow \mathbf{R}^{\geq 0}$  tale che

$$\forall x, y \in V : c_f(x, y) = c(x, y) - f(x, y)$$



- Flusso in **rosso**
- Capacità iniziale in **blu**
- Capacità residua in **viola**

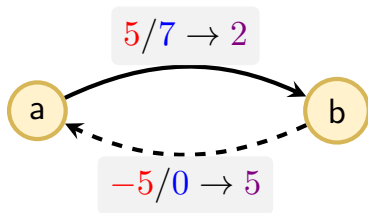
# Definizioni

## Capacità residua

Definiamo **capacità residua** di un flusso  $f$  in una rete  $G = (V, E, s, t, c)$  una funzione  $c_f : V \times V \rightarrow \mathbf{R}^{\geq 0}$  tale che

$$\forall x, y \in V : c_f(x, y) = c(x, y) - f(x, y)$$

Misura quanto possiamo ancora modificare il flusso: in avanti aumentando il flusso, all'indietro annullando flusso già inviato.



L'**arco all'indietro** non era presente inizialmente: compare perché possiamo ridurre il flusso corrente da  $a$  a  $b$ :

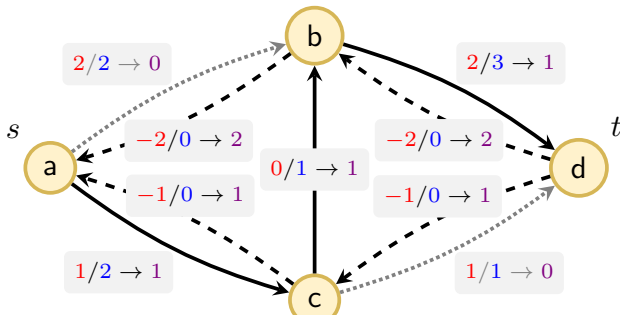
$$\begin{aligned} c_f(b, a) &= c(b, a) - f(b, a) \\ &= 0 - (-5) = 5 \end{aligned}$$

# Definizioni

## Reti residue

Data una rete di flusso  $G = (V, E, s, t, c)$  e un flusso  $f$  su  $G$ , possiamo costruire una **rete residua**  $G_f = (V, E_f, s, t, c_f)$ , tale per cui:

$$\forall x, y \in V : (x, y) \in E_f \Leftrightarrow c_f(x, y) > 0$$



# Algoritmo, schema generale

---

```
int[][] maxFlow(GRAPH  $G$ , NODE  $s$ , NODE  $t$ , int[][]  $c$ )
```

---

**precondition:**  $G = (V, E, s, t, c)$  è una rete di flusso con capacità intere non negative

```

 $f = f_0$                                 % Inizializza un flusso nullo
 $c_f = c$                                 % Capacità residua, inizialmente pari a  $c$  perché  $f = f_0$ 
repeat
     $G_f =$  rete residua associata al flusso  $f$ 
     $g =$  trova un flusso in  $G_f$  tale che  $|g| > 0$ , altrimenti  $f_0$ 
     $f = f + g$ 
     $c_f =$  Capacità residua del flusso  $f$  in  $G$ 
until  $g = f_0$ 
return  $f$ 

```

---

# Dimostrazione correttezza

## Lemma

Se  $f$  è un flusso in  $G$  e  $g$  è un flusso in  $G_f$ , allora  $f + g$  è un flusso in  $G$ .

Questo lemma giustifica l'operazione fondamentale dell'algoritmo: possiamo migliorare il flusso corrente sommando un flusso valido nella rete residua, senza violare i vincoli della rete iniziale.

# Dimostrazione correttezza

## Lemma

Se  $f$  è un flusso in  $G$  e  $g$  è un flusso in  $G_f$ , allora  $f + g$  è un flusso in  $G$ .

## Dimostrazione

- Vincolo sulla capacità?
- Antisimmetria?
- Conservazione del flusso?

## Reminder

$$G_f = (V, E_f, s, t, c_f)$$

$$\forall x, y \in V : (x, y) \in E_f \Leftrightarrow c_f(x, y) > 0$$

$$\forall x, y \in V : c_f(x, y) = c(x, y) - f(x, y)$$

# Dimostrazione correttezza

## Lemma

Se  $f$  è un flusso in  $G$  e  $g$  è un flusso in  $G_f$ , allora  $f + g$  è un flusso in  $G$ .

## Conservazione ( $\forall x \in V - \{s, t\}$ )

$$\begin{aligned}\sum_{y \in V} (f + g)(x, y) &= \sum_{y \in V} (f(x, y) + g(x, y)) \\ &= \sum_{y \in V} f(x, y) + \sum_{y \in V} g(x, y) \\ &= 0 + 0\end{aligned}$$

# Dimostrazione correttezza

## Lemma

Se  $f$  è un flusso in  $G$  e  $g$  è un flusso in  $G_f$ , allora  $f + g$  è un flusso in  $G$ .

## Antisimmetria ( $\forall x, y \in V$ )

$$f(x, y) + g(x, y) = -f(y, x) - g(y, x)$$

$$f(x, y) + g(x, y) = -(f(y, x) + g(y, x))$$

$$(f + g)(x, y) = -(f + g)(y, x)$$

Antisimmetria  $f, g$

Raccolta segno –

Sostituzione

# Dimostrazione correttezza

## Lemma

Se  $f$  è un flusso in  $G$  e  $g$  è un flusso in  $G_f$ , allora  $f + g$  è un flusso in  $G$ .

## Vincolo sulla capacità ( $\forall x, y \in V$ )

$$g(x, y) \leq c_f(x, y)$$

$g$  è un flusso in  $G_f$

$$f(x, y) + g(x, y) \leq c_f(x, y) + f(x, y) \quad \text{Aggiungo termine uguale}$$

$$(f + g)(x, y) \leq c(x, y) - f(x, y) + f(x, y) \quad \text{Sostituzione}$$

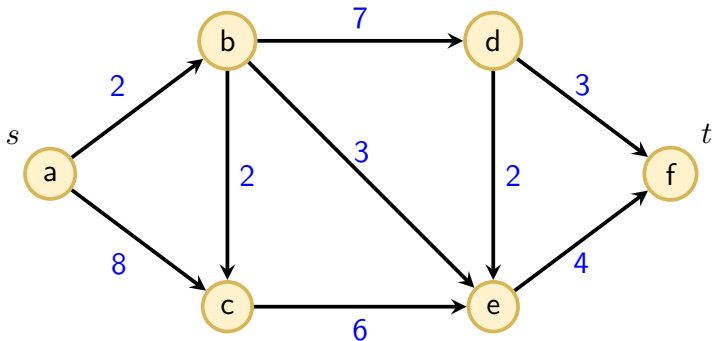
$$(f + g)(x, y) \leq c(x, y) \quad \text{Semplificazione}$$

# Flusso aggiuntivo

## Domanda

Il problema principale del metodo proposto è il seguente:

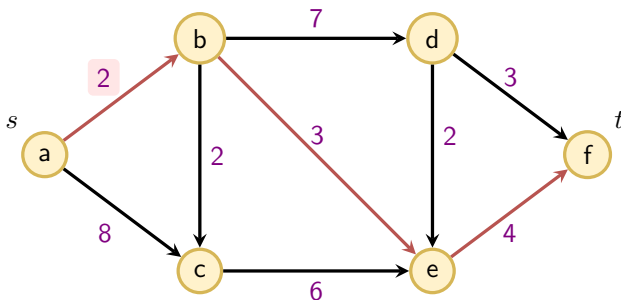
**Come trovare un flusso aggiuntivo?**



# Cammini Aumentanti: Ford-Fulkerson, 1956

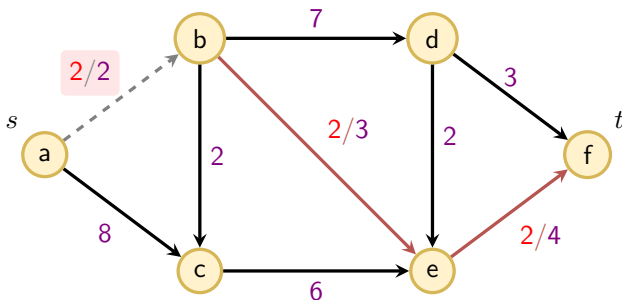
- Si trova un cammino  $C = v_0, v_1, \dots, v_k$ , con  $s = v_0$  e  $t = v_k$  nella rete residua  $G_f$ ;
- Si identifica la **capacità del cammino**, corrispondente alla minore capacità degli archi incontrati (**collo di bottiglia**):

$$c_f(C) = \min_{i=1 \dots k} c_f(v_{i-1}, v_i)$$



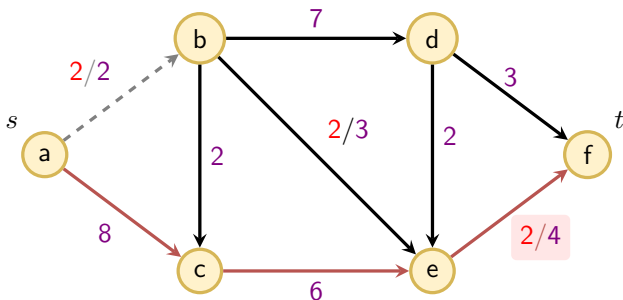
# Cammini Aumentanti: Ford-Fulkerson, 1956

- si crea un flusso addizionale  $g$  tale che
  - $g(v_{i-1}, v_i) = c_f(C)$ ;
  - $g(v_i, v_{i-1}) = -c_f(C)$  (per antisimmetria)
  - $g(x, y) = 0$  per tutte le altre coppie  $(x, y)$



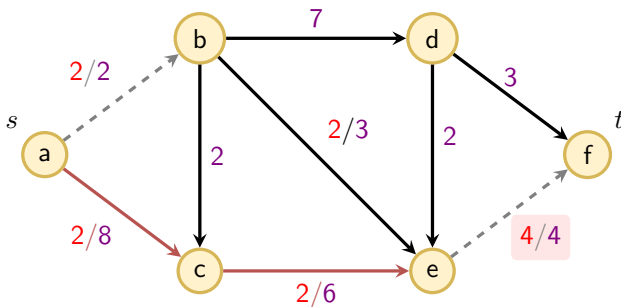
# Cammini Aumentanti: Ford-Fulkerson, 1956

- si crea un flusso addizionale  $g$  tale che
  - $g(v_{i-1}, v_i) = c_f(C)$ ;
  - $g(v_i, v_{i-1}) = -c_f(C)$  (per antisimmetria)
  - $g(x, y) = 0$  per tutte le altre coppie  $(x, y)$



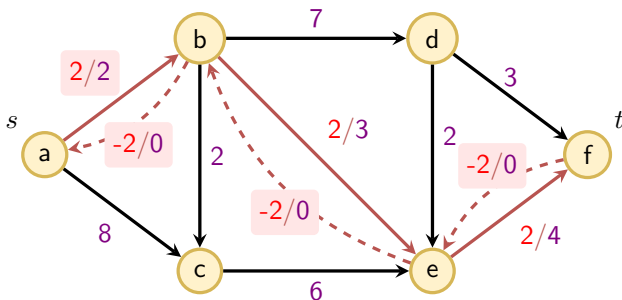
# Cammini Aumentanti: Ford-Fulkerson, 1956

- si crea un flusso addizionale  $g$  tale che
  - $g(v_{i-1}, v_i) = c_f(C)$ ;
  - $g(v_i, v_{i-1}) = -c_f(C)$  (per antisimmetria)
  - $g(x, y) = 0$  per tutte le altre coppie  $(x, y)$



# Cammini Aumentanti: Ford-Fulkerson, 1956

- si crea un flusso addizionale  $g$  tale che
  - $g(v_{i-1}, v_i) = c_f(C)$ ;
  - $g(v_i, v_{i-1}) = -c_f(C)$  (per antisimmetria)
  - $g(x, y) = 0$  per tutte le altre coppie  $(x, y)$



# Cammini Aumentanti: Ford-Fulkerson, 1956

---

```
int[][] maxFlow(GRAPH  $G$ , NODE  $s$ , NODE  $t$ , int[][]  $c$ )
```

---

**precondition:**  $G = (V, E, s, t, c)$  è una rete di flusso con capacità intere non negative

```
int[][]  $f$  = new int[][]                                % Flusso parziale
int[][]  $g$  = new int[][]                                % Flusso da cammino aumentante
int[][]  $c_f$  = new int[][]                                % Capacità residua

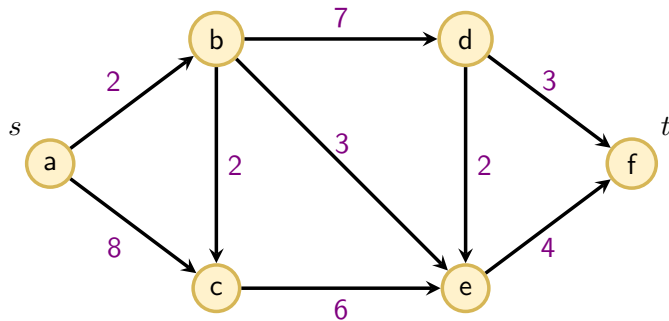
foreach  $x, y \in G.V()$  do
     $f[x][y] = 0$                                            % Inizializza un flusso nullo
     $c_f[x][y] = c[x][y]$                                      % Copia  $c$  in  $c_f$ 

repeat
     $g$  = flusso associato a un cammino aumentante in  $G_f$ , oppure  $f_0$ 
    foreach  $x, y \in G.V()$  do
         $f[x][y] = f[x][y] + g[x][y]$                        %  $f = f + g$ 
         $c_f[x][y] = c[x][y] - f[x][y]$                      % Calcola  $c_f$ 

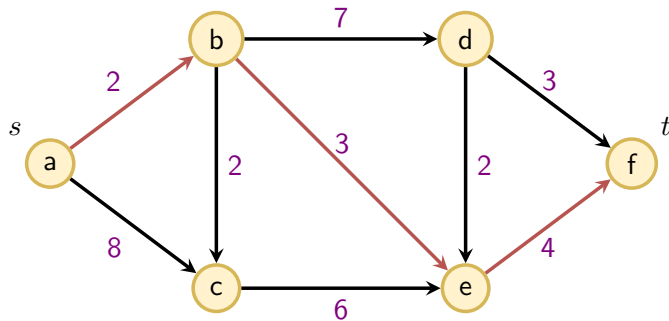
until  $g = f_0$ 
return  $f$ 
```

---

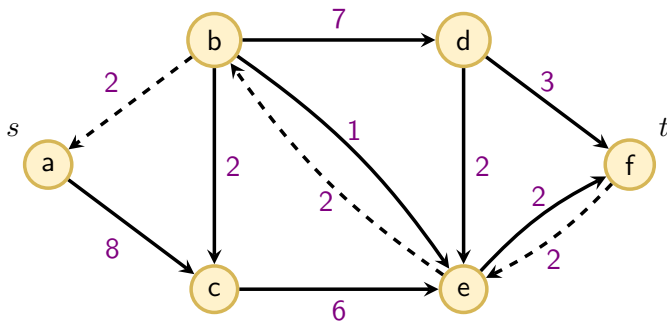
# Esecuzione



# Esecuzione



# Esecuzione

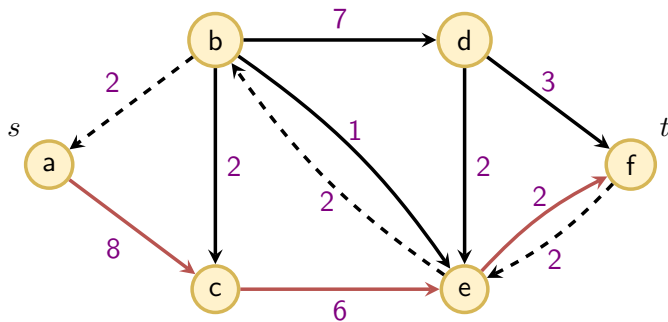


$$f(a, b) = 2$$

$$f(b, e) = 2$$

$$f(e, f) = 2$$

# Esecuzione

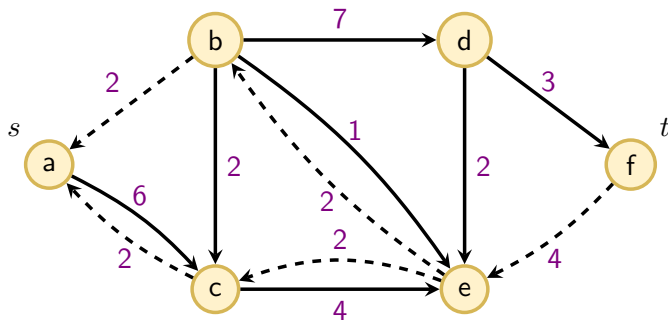


$$f(a, b) = 2$$

$$f(b, e) = 2$$

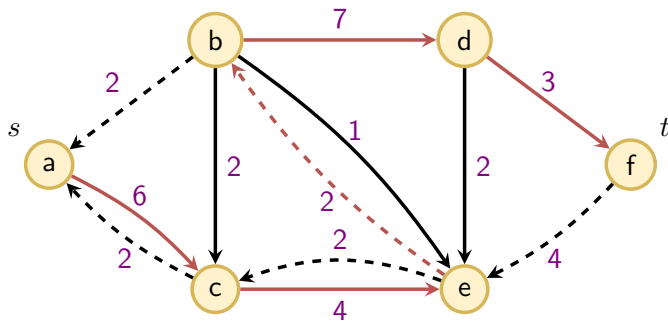
$$f(e, f) = 2$$

# Esecuzione



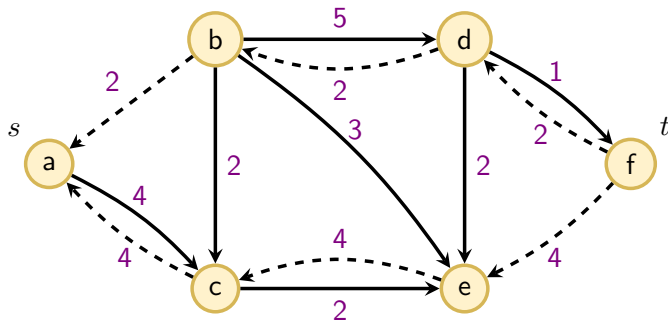
$$\begin{aligned} f(a, b) &= 2 \\ f(b, e) &= 2 \\ f(e, f) &= 4 \\ f(a, c) &= 2 \\ f(c, e) &= 2 \end{aligned}$$

# Esecuzione



$$\begin{aligned}
 f(a, b) &= 2 \\
 f(b, e) &= 2 \\
 f(e, f) &= 4 \\
 f(a, c) &= 2 \\
 f(c, e) &= 2
 \end{aligned}$$

# Esecuzione



$$f(a, b) = 2$$

$$f(b, e) = 3$$

$$f(e, f) = 4$$

$$f(a, c) = 4$$

$$f(c, e) = 2$$

$$f(b, d) = 5$$

$$f(d, f) = 1$$

## Ricerca del cammino

Ford e Fulkerson (1956) non fissano una regola specifica: è sufficiente trovare un qualunque cammino aumentante nella rete residua.

Edmonds e Karp (1972) suggerirono di utilizzare una visita in ampiezza.

Costo della visita:  $\mathcal{O}(V + E)$

# Ricerca del cammino

```
/**
 * Calcola il flusso massimo con l'algoritmo di Ford-Fulkerson.
 * @param capacity matrice delle capacita'
 * @param source nodo sorgente
 * @param sink nodo pozzo
 * @return matrice del flusso massimo
 */
private static int[][] maxFlow(int[][] capacity, int source, int sink)
// Inizializza un flusso nullo
int[][] flow = new int[capacity.length][capacity.length];
// Inizializza la rete residua come copia delle capacita'
int[][] residual = copy(capacity);
// Inizializza l'insieme dei nodi visitati
boolean[] visited = new boolean[capacity.length];
// Ripete finche' esiste un cammino aumentante
while (dfs(residual, flow, source, sink, visited,
    Integer.MAX_VALUE) > 0) {
    Arrays.fill(visited, false);
}
return flow;
}
```

# Ricerca del cammino

```

/**
 * Esegue una DFS dal nodo corrente verso il pozzo.
 * @param residual matrice delle capacita' residue
 * @param flow matrice del flusso
 * @param current nodo corrente
 * @param sink nodo pozzo
 * @param visited nodi gia' visitati
 * @param bottleneck minima capacita' incontrata
 * @return valore del flusso aggiuntivo trovato
 */
private static int dfs(int[][] residual, int[][] flow, int current,
    int sink, boolean[] visited, int bottleneck) {
    if (current == sink) return bottleneck; // Il pozzo e' stato raggiunto
    visited[current] = true;
    for (int next = 0; next < residual.length; next++) {
        if (residual[current][next] > 0 && !visited[next]) {
            int delta = dfs(residual, flow, next, sink, visited,
                Math.min(bottleneck, residual[current][next]));
            if (delta > 0) {
                residual[current][next] -= delta;
                residual[next][current] += delta;
                flow[current][next] += delta;
                flow[next][current] -= delta;
                return delta;
            }
        }
    }
    return 0; // Il pozzo non e' stato raggiunto
}

```

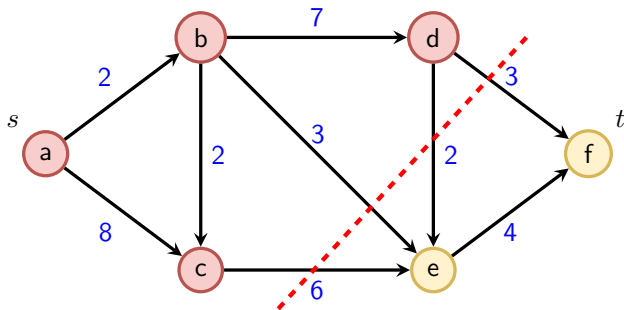
# Dimostrazione correttezza – Definizioni

## Taglio

Un **taglio**  $(S, T)$  della rete di flusso  $G = (V, E, s, t, c)$  è una partizione di  $V$  in due sottoinsiemi disgiunti  $S, T$  tali che:

$$S = V - T$$

$$s \in S \wedge t \in T$$



$$S = \{a, b, c, d\}$$

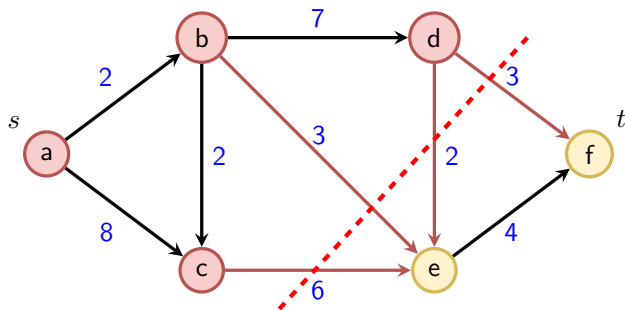
$$T = \{e, f\}$$

# Dimostrazione correttezza – Definizioni

## Capacità di un taglio

La **capacità**  $C(S, T)$  attraverso il taglio  $(S, T)$  è pari a:

$$C(S, T) = \sum_{x \in S, y \in T} c(x, y)$$



$$S = \{a, b, c, d\}$$

$$T = \{e, f\}$$

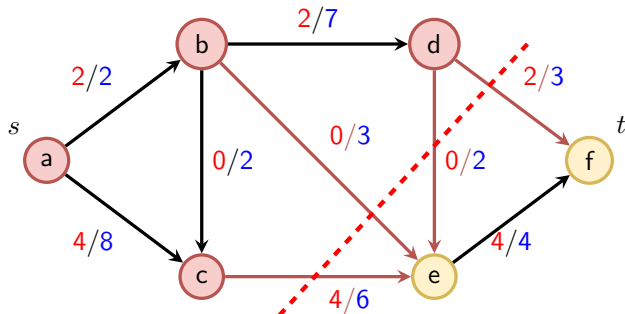
$$C(S, T) = 14$$

# Dimostrazione correttezza – Definizioni

## Flusso di un taglio

Se  $f$  è un flusso in  $G$ , il **flusso netto**  $F_f(S, T)$  attraverso  $(S, T)$  è:

$$F_f(S, T) = \sum_{x \in S, y \in T} f(x, y)$$



$$S = \{a, b, c, d\}$$

$$T = \{e, f\}$$

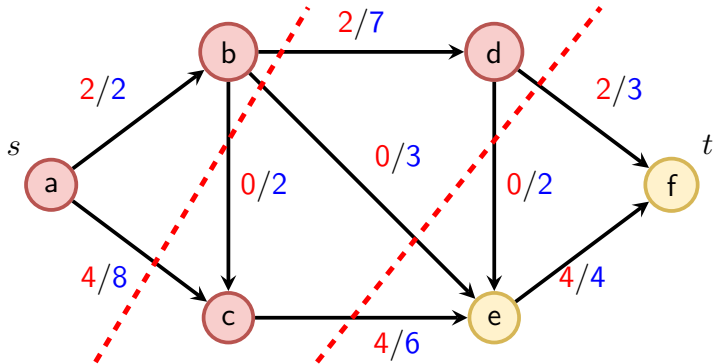
$$C(S, T) = 14$$

$$F_f(S, T) = 6$$

# Dimostrazione correttezza – Lemma valore del flusso

## Lemma – Valore del flusso di un taglio

Dato un flusso  $f$  e un taglio  $(S, T)$ , la quantità di flusso  $F_f(S, T)$  che attraversa il taglio è uguale a  $|f|$ .



# Dimostrazione correttezza – Lemma valore del flusso

## Lemma – Valore del flusso di un taglio

Dato un flusso  $f$  e un taglio  $(S, T)$ , la quantità di flusso  $F_f(S, T)$  che attraversa il taglio è uguale a  $|f|$ .

$$\begin{aligned}
 F_f(S, T) &= \sum_{x \in S, y \in T} f(x, y) \\
 &= \sum_{x \in S, y \in V} f(x, y) - \sum_{x \in S, y \in S} f(x, y) && T = V - S \\
 &= \sum_{x \in S, y \in V} f(x, y) - 0 && \text{Antisimmetria} \\
 &= [\dots]
 \end{aligned}$$

# Dimostrazione correttezza – Lemma valore del flusso

## Lemma – Valore del flusso di un taglio

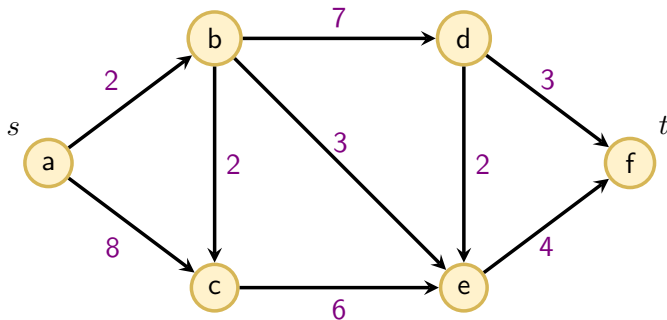
Dato un flusso  $f$  e un taglio  $(S, T)$ , la quantità di flusso  $F_f(S, T)$  che attraversa il taglio è uguale a  $|f|$ .

$$\begin{aligned}
 F_f(S, T) &= \sum_{x \in S, y \in V} f(x, y) \\
 &= \sum_{x \in S - \{s\}, y \in V} f(x, y) + \sum_{y \in V} f(s, y) && s \in S \\
 &= \sum_{x \in S - \{s\}} \sum_{y \in V} f(x, y) + \sum_{y \in V} f(s, y) && \text{Sommatoria} \\
 &= \sum_{x \in S - \{s\}} 0 + \sum_{y \in V} f(s, y) && \text{Conservazione flusso} \\
 &= \sum_{y \in V} f(s, y) = |f| && \text{Definizione valore flusso}
 \end{aligned}$$

# Dimostrazione correttezza – Lemma capacità del taglio

## Lemma – Capacità taglio

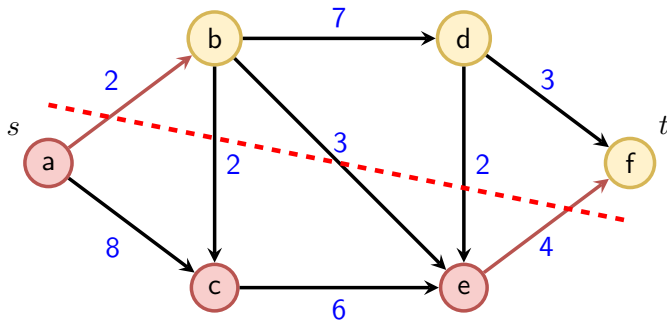
Il **flusso massimo** è limitato superiormente dalla capacità del **taglio minimo**, ovvero il taglio la cui capacità è minore fra tutti i tagli.



# Dimostrazione correttezza – Lemma capacità del taglio

## Lemma – Capacità taglio

Il **flusso massimo** è limitato superiormente dalla capacità del **taglio minimo**, ovvero il taglio la cui capacità è minore fra tutti i tagli.



# Dimostrazione correttezza – Lemma capacità del taglio

## Lemma – Capacità taglio

Il **flusso massimo** è limitato superiormente dalla capacità del **taglio minimo**, ovvero il taglio la cui capacità è minore fra tutti i tagli.

- Nessun flusso attraverso un taglio supera la capacità del taglio

$$\forall f : F_f(S, T) \leq C(S, T) \quad \forall (S, T) \text{ taglio di } V$$

Dimostrazione:

$$\forall f : F_f(S, T) = \sum_{x \in S, y \in T} f(x, y) \leq \sum_{x \in S, y \in T} c(x, y) = C(S, T)$$

(Vincolo sulla capacità)

# Dimostrazione correttezza – Lemma capacità del taglio

## Lemma – Capacità taglio

Il **flusso massimo** è limitato superiormente dalla capacità del **taglio minimo**, ovvero il taglio la cui capacità è minore fra tutti i tagli.

- Nessun flusso attraverso un taglio supera la capacità del taglio

$$\forall f : F_f(S, T) \leq C(S, T) \quad \forall (S, T) \text{ taglio di } V$$

- Il flusso che attraversa un taglio è uguale al valore del flusso

$$\forall f : |f| = F_f(S, T) \quad \forall (S, T) \text{ taglio di } V$$

- Quindi, il valore del flusso è limitato superiormente dalla capacità di tutti i possibili tagli.

$$\forall f : |f| \leq C(S, T) \quad \forall (S, T) \text{ taglio di } V$$

# Teorema del taglio minimo / flusso massimo

## Teorema

Le seguenti tre affermazioni sono equivalenti:

- ①  $f$  è un **flusso massimo**
- ② non esiste alcun cammino aumentante nella rete residua  $G_f$
- ③ esiste un **taglio minimo**  $(S, T)$  tale che  $C(S, T) = |f|$

Dimostreremo circolarmente:

- $(1) \Rightarrow (2)$
- $(2) \Rightarrow (3)$
- $(3) \Rightarrow (1)$

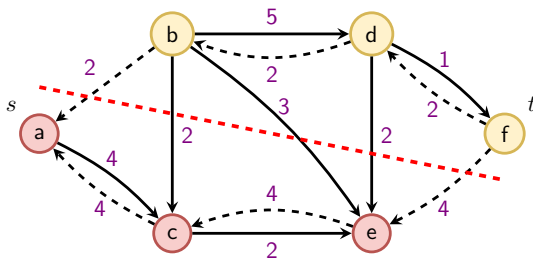
## Dimostrazione correttezza – (1) $\Rightarrow$ (2)

$f$  è un flusso massimo  $\Rightarrow$   
non esiste nessun cammino aumentante nella rete residua  $G_f$

- Se esistesse un cammino aumentante, il flusso potrebbe essere aumentato e quindi non sarebbe massimo (assurdo).

## Dimostrazione correttezza – $(2) \Rightarrow (3)$

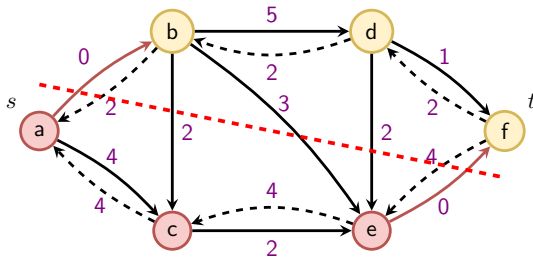
Non esiste nessun cammino aumentante nella rete residua  $G_f \Rightarrow$  esiste un taglio minimo  $(S, T)$  tale che  $C(S, T) = |f|$



- Poiché non esiste nessun cammino aumentante nella rete residua  $G_f$ , non esiste nessun cammino da  $s$  a  $t$
- Sia  $S$  l'insieme dei vertici raggiungibili da  $s$  in  $G_f$ ;  $T = V - S$
- Ovviamente  $s \in S$  e  $t \in T$ , quindi  $(S, T)$  è un taglio

## Dimostrazione correttezza – $(2) \Rightarrow (3)$

Non esiste nessun cammino aumentante nella rete residua  $G_f \Rightarrow$  esiste un taglio minimo  $(S, T)$  tale che  $C(S, T) = |f|$



- Poiché  $t$  non è raggiungibile da  $s$  in  $G_f$ , tutti gli archi  $(x, y)$  con  $x \in S$  e  $y \in T$  sono saturati; ovvero,  $f(x, y) = c(x, y)$ .

## Dimostrazione correttezza – (2) $\Rightarrow$ (3)

Non esiste nessun cammino aumentante nella rete residua  $G_f \Rightarrow$  esiste un taglio minimo  $(S, T)$  tale che  $C(S, T) = |f|$

- Per il Lemma – Valore del flusso di un taglio,

$$|f| = \sum_{x \in S, y \in T} f(x, y)$$

- Ne segue che:

$$|f| = \sum_{x \in S, y \in T} f(x, y) = \sum_{x \in S, y \in T} c(x, y) = C(S, T)$$

- $(S, T)$  è minimo perché  $|f| = C(S, T)$  e per ogni taglio  $(S', T')$ , abbiamo che  $|f| \leq C(S', T')$  (per il lemma sulla capacità del taglio)

## Dimostrazione correttezza – (3) $\Rightarrow$ (1)

Esiste un taglio  $(S, T)$  tale che  $C(S, T) = |f| \Rightarrow$   
 $f$  è un flusso massimo

Poiché per un qualsiasi flusso  $f$  e un qualsiasi taglio  $(S, T)$  vale la relazione  $|f| \leq C(S, T)$ , il flusso che soddisfa  $|f| = C(S, T)$  deve essere massimo.

# Complessità

## Complessità, limite superiore – Ford-Fulkerson

Se le capacità sono **interi**, l'algoritmo di Ford-Fulkerson ha complessità  $\mathcal{O}((V + E)|f^*|)$  (liste) o  $\mathcal{O}(V^2|f^*|)$  (matrice).

- L'algoritmo parte dal flusso nullo e termina quando il valore totale del flusso raggiunge  $|f^*|$
- Ogni incremento del flusso aumenta il flusso di almeno un'unità
- Ogni ricerca di un cammino richiede una visita del grafo, con costo  $\mathcal{O}(V + E)$  o  $\mathcal{O}(V^2)$ ;
- La somma dei flussi e il calcolo della rete residua può essere effettuato in tempo  $\mathcal{O}(V + E)$  o  $\mathcal{O}(V^2)$ .

# Complessità

## Complessità, limite superiore – Edmonds e Karp

L'algoritmo di Edmonds–Karp ha complessità  $\mathcal{O}(VE^2)$  nel caso pessimo, indipendentemente dai valori numerici delle capacità.

Come si conciliano i due limiti superiori?

- $\mathcal{O}(VE^2)$  vs  $\mathcal{O}((V + E)|f^*|)$
- Sono entrambi limiti superiori
- Sono entrambi validi
- Si deve quindi prendere il più basso fra i due

# Dimostrazione complessità

## Teorema

La complessità dell'algoritmo di Edmonds-Karp è  $\mathcal{O}(VE^2)$ .

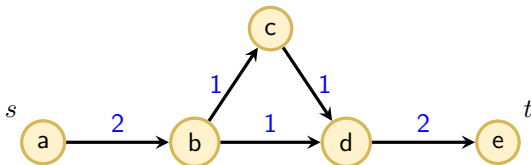
- Vengono eseguiti  $\mathcal{O}(VE)$  aumenti di flusso, ognuno dei quali richiede una visita in ampiezza  $\mathcal{O}(V + E)$ .
- La complessità è quindi  $\mathcal{O}(VE(V + E))$ .
- Poiché  $E = \Omega(V)$  (ogni nodo diverso da sorgente, pozzo ha almeno un arco entrante e un arco uscente), possiamo semplificare scrivendo che la complessità è  $\mathcal{O}(VE^2)$

# Dimostrazione complessità

## Lemma - Monotonia

Nell'algoritmo di Edmonds-Karp, sia  $\delta_f(s, x)$  la distanza minima da  $s$  a  $x$  nella rete residua  $G_f$ . Se  $f'$  è ottenuto da  $f$  aumentando il flusso lungo un cammino aumentante minimo in  $G_f$ , allora  $\delta_{f'}(s, x) \geq \delta_f(s, x)$  per ogni nodo  $x$  raggiungibile da  $s$  in entrambe le reti residue.

- Quando viene aumentato il flusso, alcuni archi si “spengono” (capacità residua 0)
- Questi archi erano utilizzati nei cammini minimi (BFS)
- I cammini minimi non possono diventare più corti

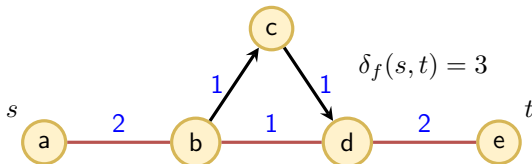


# Dimostrazione complessità

## Lemma - Monotonia

Nell'algoritmo di Edmonds-Karp, sia  $\delta_f(s, x)$  la distanza minima da  $s$  a  $x$  nella rete residua  $G_f$ . Se  $f'$  è ottenuto da  $f$  aumentando il flusso lungo un cammino aumentante minimo in  $G_f$ , allora  $\delta_{f'}(s, x) \geq \delta_f(s, x)$  per ogni nodo  $x$  raggiungibile da  $s$  in entrambe le reti residue.

- Quando viene aumentato il flusso, alcuni archi si “spengono” (capacità residua 0)
- Questi archi erano utilizzati nei cammini minimi (BFS)
- I cammini minimi non possono diventare più corti

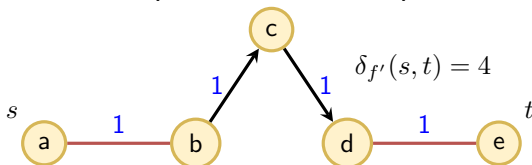


# Dimostrazione complessità

## Lemma - Monotonia

Nell'algoritmo di Edmonds-Karp, sia  $\delta_f(s, x)$  la distanza minima da  $s$  a  $x$  nella rete residua  $G_f$ . Se  $f'$  è ottenuto da  $f$  aumentando il flusso lungo un cammino aumentante minimo in  $G_f$ , allora  $\delta_{f'}(s, x) \geq \delta_f(s, x)$  per ogni nodo  $x$  raggiungibile da  $s$  in entrambe le reti residue.

- Quando viene aumentato il flusso, alcuni archi si “spengono” (capacità residua 0)
- Questi archi erano utilizzati nei cammini minimi (BFS)
- I cammini minimi non possono diventare più corti



# Dimostrazione complessità

## Lemma - Aumenti di flusso

Il numero totale di aumenti di flusso eseguiti dall'algoritmo di Edmonds e Karp è  $\mathcal{O}(VE)$ .

- Sia  $G_f$  una rete residua
- Sia  $C$  un cammino aumentante di  $G_f$ .
- $(x, y)$  è un arco **critico** (collo di bottiglia) in  $C$  se

$$c_f(x, y) = \min_{(u,v) \in C} \{c_f(u, v)\}$$

- In ogni cammino esiste almeno un arco critico
- Una volta aggiunto il flusso associato a  $C$ , l'arco critico scompare dalla rete residua.

# Dimostrazione complessità

- Poiché i cammini aumentanti sono cammini minimi, abbiamo che:

$$\delta_f(s, y) = \delta_f(s, x) + 1$$

- L'arco  $(x, y)$  potrà ricomparire se e solo se il flusso lungo l'arco diminuirà, ovvero se  $(y, x)$  appare in un cammino aumentante
- Sia  $g$  il flusso quando questo accade; come sopra, abbiamo:

$$\delta_g(s, x) = \delta_g(s, y) + 1$$

- Per Lemma (Monotonia), abbiamo anche che  $\delta_g(s, y) \geq \delta_f(s, y)$ ; quindi:

$$\begin{aligned} \delta_g(s, x) &= \delta_g(s, y) + 1 \\ &\geq \delta_f(s, y) + 1 \\ &= \delta_f(s, x) + 2 \end{aligned}$$

# Dimostrazione complessità

- Dal momento in cui un arco  $(x, y)$  è critico al momento in cui può tornare ad essere critico, la distanza minima da  $s$  al suo estremo iniziale  $u$  si è allungata di almeno due passi.
- La lunghezza massima del cammino fino a  $x$ , tenuto conto che poi si deve ancora seguire l'arco  $(x, y)$ , è  $V - 2$ .
- Quindi un arco può diventare critico al massimo  $(V - 2)/2 = V/2 - 1$  volte.
- Poiché ci sono  $\mathcal{O}(E)$  archi che possono diventare critici  $\mathcal{O}(V)$  volte, abbiamo che il numero massimo di flussi aumentanti è  $\mathcal{O}(VE)$ .

# Complessità – Altre versioni

| Nome                    | Anno | Complessità                                                         | Note                                             |
|-------------------------|------|---------------------------------------------------------------------|--------------------------------------------------|
| Ford–Fulkerson          | 1956 | $\mathcal{O}(E f^* )$                                               | Per capacità intere; pseudo-polinomiale          |
| Edmonds–Karp            | 1972 | $\mathcal{O}(VE^2)$                                                 | Specializzazione di Ford–Fulkerson basata su BFS |
| Dinitz, blocking flow   | 1970 | $\mathcal{O}(V^2E)$                                                 | Usa reti stratificate e flussi bloccanti         |
| Dinitz su reti unitarie | 1970 | $\mathcal{O}(\min(V^{2/3}, E^{1/2})E)$                              | Migliora il bound su reti particolari            |
| MPM                     | 1978 | $\mathcal{O}(V^3)$                                                  | Usa reti stratificate acicliche                  |
| Sleator–Tarjan          | 1983 | $\mathcal{O}(VE \log V)$                                            | Dinitz con dynamic trees                         |
| Goldberg–Tarjan         | 1986 | $\mathcal{O}(VE \log(V^2/E))$                                       | Algoritmi preflow-push con dynamic trees         |
| Goldberg–Rao            | 1998 | $\mathcal{O}(\min(V^{2/3}, E^{1/2})E \cdot \log(V^2/E + 2) \log C)$ | $C = \max_{(x,y) \in E} c(x, y)$                 |
| Orlin                   | 2013 | $\mathcal{O}(VE)$                                                   | Algoritmo fortemente polinomiale                 |
| Chen et al.             | 2022 | $E^{1+o(1)} \cdot \text{polylog}(C)$                                | Quasi-lineare                                    |

# Complessità – Letture

Andrew V. Goldberg, Robert E. Tarjan. [Efficient Maximum Flow Algorithms](#). Communications of the ACM, 57(8):82–89. 2014.

Li Chen, Rasmus Kyng, Yang P. Liu, Richard Peng, Maximilian Probst Gutenberg, Sushant Sachdeva. [Maximum Flow and Minimum-Cost Flow in Almost-Linear Time](#). 63rd IEEE Annual Symposium on Foundations of Computer Science, FOCS 2022, Denver, CO, USA, October 31 - November 3, 2022.

# Applicazioni

- **Maximum Flows with Edge Demands (Circulation Problem)**

Flusso massimo con limiti inferiori e superiori per la capacità

- **Node Supplies and Demands**

Il flusso può essere generato o consumato in ogni nodo

- **Min-Cost Max Flows**

Al flusso è associato un costo, fra due flussi massimali scegliamo quello con costo minimo

Jeff Erickson. [Lecture 18: Extensions of Maximum Flow](#) (8 pp.)

David P. Williamson. [Network Flow Algorithms](#).  
Cambridge University Press, 2019 (273 pp.)

# Applicazioni

- Bipartite matching
- Data mining
- Project selection
- Airline scheduling
- Baseball elimination
- Image segmentation
- Network connectivity
- Network reliability
- Distributed computing
- Egalitarian stable matching
- Security of statistical data
- Network intrusion detection
- Multi-camera scene reconstruction
- Gene function prediction

Jeff Erickson. [Lecture 11: Applications of Flows and Cuts.](#)

# Abbinamento (matching) massimo nei grafi bipartiti

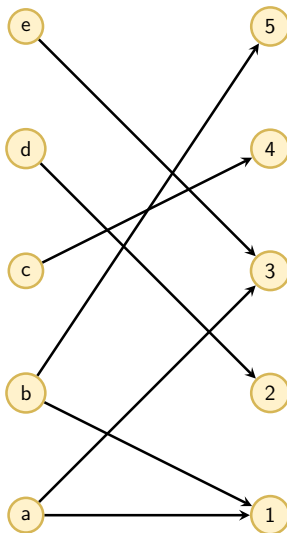
## Problema - Job Assignment - Input

- Un insieme  $J$  contenente  $n$  job
- Un insieme  $W$  contenente  $m$  worker
- Una relazione  $R \subseteq J \times W$ , tale che  $(j, w) \in R$  se e solo se il job  $j$  può essere eseguito dal worker  $w$

## Problema - Job Assignment - Output

- Il più grande sottoinsieme  $O \subseteq R$ , tale che:
  - ogni job venga assegnato al più a un worker
  - ad ogni worker venga assegnato al più un job

# Esempio



# Esempio

